SPI_Slave Verification and RTL2GDS Flow

Koen Van Caekenberghe, ChipDesign B.V.

June 2026

Contents

1 SPI Slave Description

1.1 Overview

The spi_slave is a small, synchronous SPI slave IP block designed for use in the IHP SG13G2 process. It exposes eight 8-bit configuration and status registers to an SPI master and provides a secondary parallel read port for on-chip register readback independent of the SPI bus.

1.2 SPI Protocol

The core operates in **SPI Mode 0** (CPOL = 0, CPHA = 0): the master drives data on the falling edge of SCK and the slave samples on the rising edge.

Every transaction consists of two bytes:

1. **Command byte** — the MSB (bit 7) carries the R/W flag: 1 = Read, 0 = Write; bits [6:0] carry the 7-bit register address.
2. **Data byte** — for a write this is the value to store; for a read the slave shifts out the register value on MISO while MOSI is ignored.

1.3 Register Map

Eight 8-bit registers are addressable at addresses 0x00–0x07. A read at any address outside this range returns 0xFF. Reset (power-on) values are listed below.

Address	Register	Reset value	Address	Register	Reset value
0x00	Register0	0x0A	0x04	Register4	0x0E
0x01	Register1	0x0B	0x05	Register5	0x10
0x02	Register2	0x0C	0x06	Register6	0x20
0x03	Register3	0x0D	0x07	Register7	0x30

1.4 Port Interface

Port	Direction	Description
Clk	in	System clock; all internal logic is synchronous to this domain
iRST_N	in	Active-low asynchronous reset
SCK	in	SPI clock from the master
MOSI	in	Master-out slave-in data
SSEL	in	SPI slave select, active low
MISO	out	Master-in slave-out data
Add2_in	in	Parallel read port: 8-bit register address
Data2_out	out	Parallel read port: register data (updated every Clk cycle)
debug	out	Sticky OR of state-machine states visited since last reset

1.5 Internal State Machine

The SPI interface logic is implemented as a Moore state machine with five states clocked by Clk. Asynchronous SPI inputs (SCK, MOSI, SSEL) are first captured through a two-flip-flop metastability synchroniser before entering the state machine.

State	Description	debug bit
IDLE	Waits for SSEL to go low	0
COMMAND	Shifts in 8-bit command/address byte on SCK rising edges	1
WRITE	Shifts in 8-bit data byte; writes register on completion	2
READ	Shifts out register value on MISO on SCK falling edges	3
END	Waits for SSEL to return high, then returns to IDLE	4

2 RTL2GDS Flow

The repository includes a **LibreLane**-based RTL2GDS flow configured for the IHP SG13G2 PDK.

2.1 Key Flow Files

- flow/Makefile – entry point for LibreLane and OpenROAD commands.
- flow/config.yaml – LibreLane configuration file.
- flow/run_flow.sh – environment setup and flow wrapper.
- flow/ihp_pdk.env.example – example PDK environment template.
- flow/constraint.sdc – timing constraint for the spi_slave clock.
- flow/place_route.tcl – fallback OpenROAD placement/routing script.
- flow/spi_slave_synth.v – Yosys-generated structural Verilog netlist (used in B2B verification).

2.2 Running the Flow

```
cd spi_slave_github_repo/flow
cp ihp_pdk.env.example ihp_pdk.env
# Edit ihp_pdk.env to point to the local IHP PDK installation
./run_flow.sh
```

2.3 Flow Execution Results

The RTL2GDS flow was executed from flow/ and produced a completed LibreLane/OpenROAD run under runs/RUN_2026-06-07_13-47-22. Key outputs include:

- runs/RUN_2026-06-07_13-47-22/final/gds/spi_slave.gds (803 KiB, includes seal ring)
- runs/RUN_2026-06-07_13-47-22/final/def/spi_slave.def (624 KiB)

The flow ran all 69 steps and completed with warnings only (no errors). A patch was applied to the installed librelane seal-ring script to fix a KLayout API incompatibility with the PDK's Python-wrapper PCells: layout.create_cell was replaced by add_pcell_variant and the die dimensions were corrected from database units (nm) to the μm values expected by the seal-ring PCell.

3 Back-to-Back Verification

3.1 Methodology

Back-to-back (B2B) verification confirms functional equivalence between the RTL source (spi_slave.v) and the Yosys-synthesised netlist (spi_slave_synth.v, in flow/). Both receive identical stimulus and their outputs are compared.

The testbench tb_spi_slave.v drives two SPI transactions:

1. **READ** from Register 1 (address byte 0x81: R/W = 1, addr = 0x01). The default reset value 0x0B is expected on MISO.
2. **WRITE** 0xFF to Register 3 (address byte 0x03: R/W = 0, addr = 0x03).

The same testbench is compiled twice: once against the RTL (spi_slave.v) and once with the synthesised netlist substituted in place of the ‘include.

3.2 Simulation Commands

```
# RTL simulation
iverilog -g2012 -o tb_rtl.out tb_spi_slave.v
vvp tb_rtl.out          # produces tb_spi_slave.vcd

# Synthesised netlist simulation (replace include at compile time)
sed 's|`include "spi_slave.v"|`include "flow/spi_slave_synth.v"|g' \
    tb_spi_slave.v \
  | sed 's|tb_spi_slave.vcd|tb_spi_slave_synth.vcd|g' \
  > tb_spi_slave_synth_run.v
iverilog -g2012 -o tb_synth.out tb_spi_slave_synth_run.v
vvp tb_synth.out  # produces tb_spi_slave_synth.vcd
```

3.3 Results

All observed signal values at the end of simulation (t = 750 ns) were identical between RTL and synthesised netlist:

Signal	RTL	Synth	Status
rst_n	1	1	MATCH
ssel	0	0	MATCH
sck	0	0	MATCH
mosi	1	1	MATCH
miso	1	1	MATCH
data	0xFF	0xFF	MATCH
debug	0x1F	0x1F	MATCH

The debug value 0x1F (bits 0–4 all set) confirms that all five state-machine states (IDLE, COMMAND, WRITE, READ, END) were visited during the simulation. The data output 0xFF confirms the WRITE transaction successfully stored 0xFF in Register 3, which is then reflected on the parallel read port (Add2_in = 3). The miso value 1 corresponds to the LSB of Register 1 (0x0B), which was the last bit shifted out during the READ transaction.

3.4 RTL Simulation Waveform

Figure 1 shows the key SPI signals, parallel data output, and debug register over the full 750 ns simulation. The two SPI transactions are visible: the READ phase (Tx1, green background) in which MISO carries Register 1's value, followed by the WRITE phase (Tx2, orange background) in which MOSI carries 0xFF to Register 3.

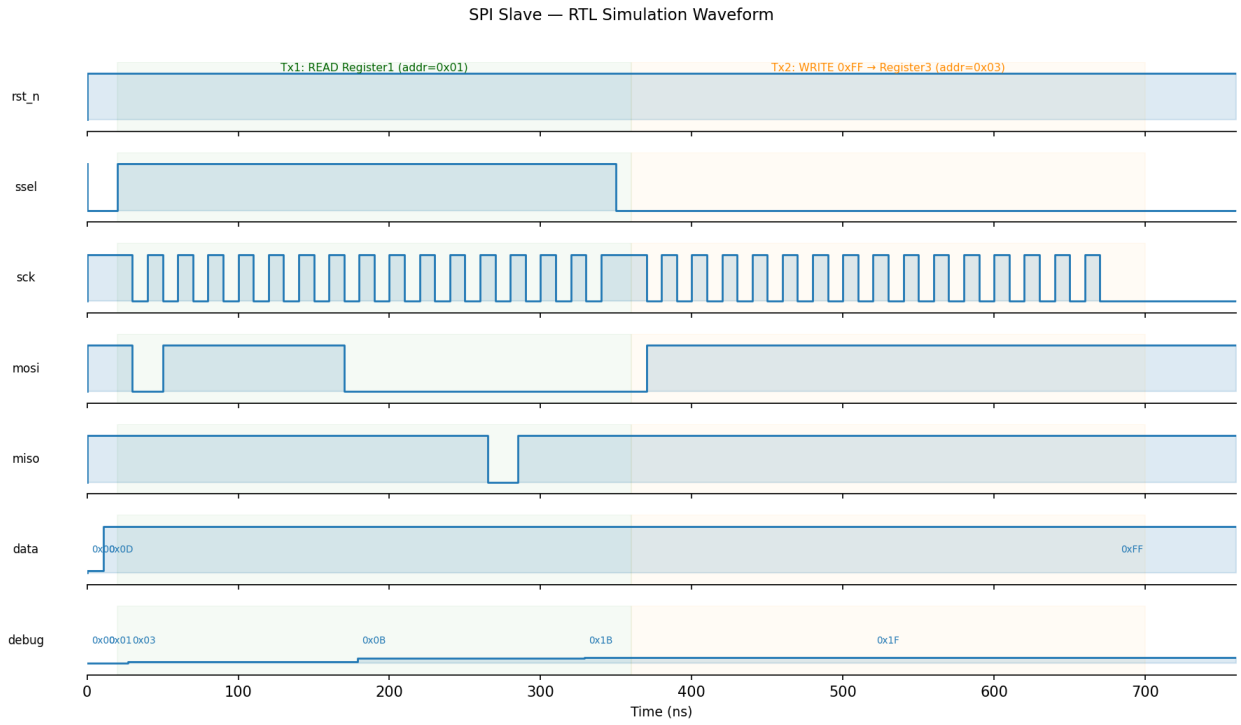

Figure 1: RTL simulation: key signals over the full testbench run.

3.5 B2B Overlay Waveform

Figure 2 overlays the RTL (blue, solid) and synthesised-netlist (red, dashed) waveforms for the output signals. The traces are indistinguishable, confirming functional equivalence.

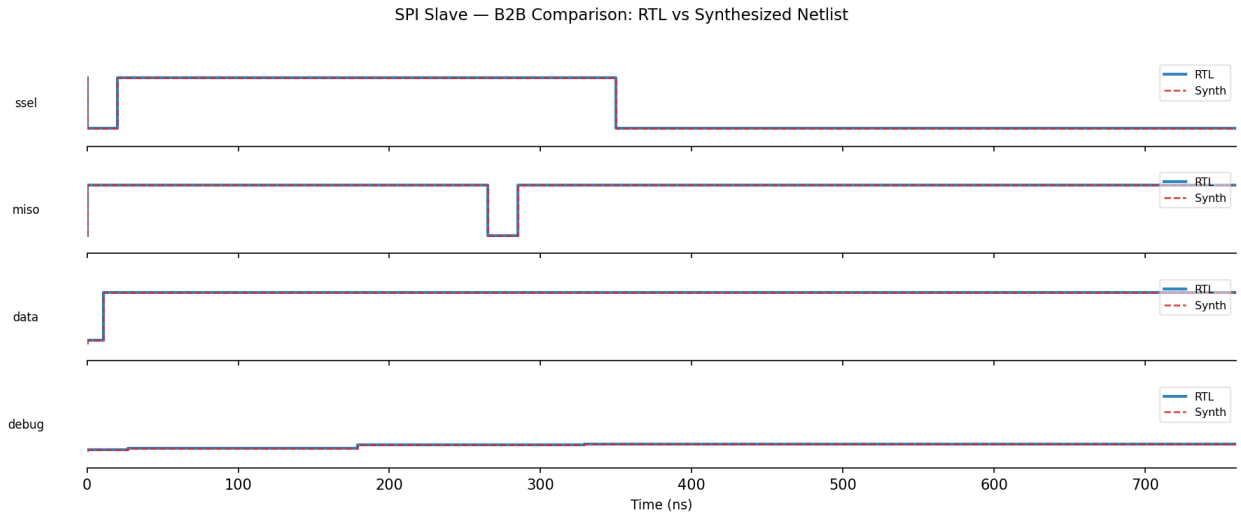

Figure 2: B2B overlay: RTL (blue solid) vs. synthesised netlist (red dashed). Traces overlap completely.

4 Conclusions

The spi_slave design has been verified at two levels:

- **Functional correctness**: Both a register read and a register write transaction were exercised in simulation, producing the expected MISO output and register update.
- **Synthesis equivalence**: All observable output signals are bit-for-bit identical between the RTL model and the Yosys-generated structural netlist, confirming that synthesis introduced no functional changes.

The LibreLane/OpenROAD RTL2GDS flow has been executed successfully, yielding a final GDS for the IHP SG13G2 target process.